Pashov Audit Group

YuzuUSD Security Review

January 14th 2026 - January 18th 2026

Contents

1. About Pashov Audit Group 3
2. Disclaimer 3
3. Risk Classification 3
4. About YuzuUSD 4
5. Executive Summary 4
6. Findings 5

Medium findings 6

[M-01] Interface mismatch in `IYuzuILPV2` and `IPSM` can lead to call revert 6
[M-02] Psm._redeem() bypasses `yzUSD` pause and redeem restrictions during execution 7

Low findings 9

[L-01] Dos in `withdrawLiquidity` due to frontrunning via instant withdrawals 9
[L-02] Dos vulnerability in `getPendingOrderIds` due to large `_pendingOrderIds` size 9
[L-03] Emergency pause does not allow fund retrieval 10
[L-04] Parameter mismatch while throwing `UnauthorizedOrderFinalizer` error 10
[L-05] Missing prechecks in `PSM.createRedeemOrder` allow unfillable redeem orders 11
[L-06] Psm lacks `maxDeposit` and `maxRedeem` preventing users from querying limits 12
[L-07] Forced cancellations are limited by the `whenNotPaused` modifier 12
[L-08] Potential DoS in `YuzuILPV2` due to unconsolidated redeem order fill 13
[L-09] EIP-712 domain version not updated to V2 allowing old signatures to remain valid 14

1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over $100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work here or reach out on Twitter @pashovkrum.

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. About YuzuUSD

YuzuUSD is a stable ERC-20 token backed 1:1 by USDC and serves as the core unit of value across the Yuzu protocol. The ecosystem includes StakedYuzuUSD, an ERC-4626 vault for staking YuzuUSD, and YuzuILP, an ERC-20 token representing deposits in the Insurance Liquidity Pool, with minting, redemption, and staking mechanisms designed to balance liquidity, yield, and risk.

5. Executive Summary

A time-boxed security review of the **Telos-Consilium/ouroboros-contracts** repository was done by Pashov Audit Group, during which **Blade, IvanFitro, merlinboii, Tejas Warambhe** engaged to review **YuzuUSD**. A total of **11** issues were uncovered.

Protocol Summary	
Project Name	YuzuUSD
Protocol Type	Stablecoin
Timeline	January 14th 2026 - January 18th 2026

Review commit hash:
- db49243c22be12e1c0068192bd3417967fd28c8d (Telos-Consilium/ouroboros-contracts)

Fixes review commit hash:
- f73fd9ae24dc550ce69e74e3a487e55109af7a37 (Telos-Consilium/ouroboros-contracts)

Scope

`IYuzuIssuer.sol` `IYuzuIssuerDefinitions.sol` `IYuzuOrderBookDefinitions.sol` `IYuzuProto.sol` `IYuzuProtoDefinitions.sol` `IPSM.sol` `IPSMDefinitions.sol` `IStakedYuzuUSD.sol` `IStakedYuzuUSDDefinitions.sol` `IYuzuILP.sol` `IYuzuILPDefinitions.sol` `IYuzuUSD.sol` `ProtoOVaultComposer.sol` `PSMOVaultComposer.sol` `YuzuIssuer.sol` `YuzuOrderBook.sol` `YuzuProto.sol` `YuzuProtoV2.sol` `PSM.sol` `StakedYuzuUSD.sol` `StakedYuzuUSDV2.sol` `YuzuILP.sol` `YuzuILPV2.sol` `YuzuUSDV2.sol`

6. Findings

Findings count

Severity	Amount
Medium	2
Low	9
Total findings	**11**

Summary of findings

ID	Title	Severity	Status
[M-01]	Interface mismatch in `IYuzuILPV2` and `IPSM` can lead to call revert	Medium	Resolved
[M-02]	Psm._redeem() bypasses `yzUSD` pause and redeem restrictions during execution	Medium	Resolved
[L-01]	Dos in `withdrawLiquidity` due to frontrunning via instant withdrawals	Low	Resolved
[L-02]	Dos vulnerability in `getPendingOrderIds` due to large `_pendingOrderIds` size	Low	Resolved
[L-03]	Emergency pause does not allow fund retrieval	Low	Acknowledged
[L-04]	Parameter mismatch while throwing `UnauthorizedOrderFinalizer` error	Low	Resolved
[L-05]	Missing prechecks in `PSM.createRedeemOrder` allow unfillable redeem orders	Low	Resolved
[L-06]	Psm lacks `maxDeposit` and `maxRedeem` preventing users from querying limits	Low	Resolved
[L-07]	Forced cancellations are limited by the `whenNotPaused` modifier	Low	Resolved
[L-08]	Potential DoS in `YuzuILPV2` due to unconsolidated redeem order fill	Low	Resolved
[L-09]	EIP-712 domain version not updated to V2 allowing old signatures to remain valid	Low	Resolved

Medium findings

[M-01] Interface mismatch in `IYuzuILPV2` and `IPSM` can lead to call revert

Severity

Impact: Low

Likelihood: High

Description

In `IYuzuILPV2.sol`, it is observed that `terminateDistribution()` takes a `receiver` address parameter:

```
interface IYuzuILPV2 is IYuzuILP {
    function lastDistributedAmount() external view returns (uint256);
    function lastDistributionPeriod() external view returns (uint256);
    function lastDistributionTimestamp() external view returns (uint256);

    function distribute(uint256 assets, uint256 period) external;
    function terminateDistribution(address receiver) external;             <<@
}
```

However, the actual `YuzuILPV2::terminateDistribution()` call lacks any such parameters:

```
    /// @notice Terminate an in-progress distribution
    function terminateDistribution() external onlyRole(POOL_MANAGER_ROLE) {
        uint256 elapsedTime = block.timestamp - lastDistributionTimestamp;
        if (lastDistributionTimestamp == 0 || elapsedTime >= lastDistributionPeriod) {
```

This can lead to failed termination calls by the pool manager if they intend to use the `IYuzuILPV2` interface.

Similarly, `IPSM::initialize()` lacks the `_minRedeemOrder` parameter, which is present in `PSM.sol`:

```
interface IPSM {
    function initialize(IERC20 __asset, IERC4626 __vault0, IERC4626 __vault1, address
_admin) external;
```

```
    function initialize(IERC20 __asset, IERC4626 __vault0, IERC4626 __vault1, address
_admin, uint256 _minRedeemOrder)
        external
        initializer
    {
```

Recommendations

It is recommended to remove the `receiver` parameter from `IYuzuILPV2::terminateDistribution()` and add the `_minRedeemOrder` parameter in `IPSM::initialize()`.

[M-02] Psm._redeem() bypasses `yzUSD` pause and redeem restrictions during execution

Severity

Impact: Medium

Likelihood: Medium

Description

The `PSM._redeem()` allows users to redeem `syzUSD` shares for underlying assets by internally burning `yzUSD` tokens. However, this implementation bypasses `yzUSD`'s pause state and redeem restriction checks.

The function is called from two entry points:

- `PSM.redeem()`, callable by users with `USER_ROLE` to instantly redeem.
- `PSM.fillRedeemOrders()`, callable by `ORDER_FILLER_ROLE` to fill orders created by users using `PSM.createRedeemOrder()`

PSM.sol#L258-L277

```
function _redeem(address caller, address _owner, address receiver, uint256 shares) internal
returns (uint256) {
    uint256 assets1 = _vault1.redeem(shares, address(this), _owner);

@>  uint256 assets0 = _vault0.convertToAssets(assets1);
@>  IERC20Burnable(address(_vault0)).burn(assets1);

    SafeERC20.safeTransfer(IERC20(asset()), receiver, assets0);
    // slither-disable-next-line reentrancy-events
    emit Withdraw(caller, receiver, _owner, assets0, shares);
    return assets0;
}
```

By burning `yzUSD` directly, `PSM._redeem()` bypasses the checks normally enforced by `YuzuProto.maxRedeem()`, which include:

- Pause enforcement
- Redeem restriction enforcement via `REDEEMER_ROLE`

As a result, users can redeem through `PSM` even when direct redemptions via `yzUSD.redeem()` would be disallowed.

YuzuProto.sol#L191-L199

```
function maxRedeem(address _owner) public view virtual override returns (uint256) {
    if (paused()) {
        return 0;
    }
    if (!_canRedeem(_owner)) {
        return 0;
    }
    return super.maxRedeem(_owner);
}
```

YuzuProto.sol#L401-L408

```
function _canRedeem(address _owner) internal view virtual returns (bool) {
    if (isRedeemRestricted) {
        if (!hasRole(REDEEMER_ROLE, _owner)) {
            return false;
        }
    }
    return true;
}
```

Note: The bypass is only effective when `syzUSD` is not paused. If `syzUSD` is paused, the call to `syzUSD.redeem()` reverts before reaching the burn logic.

Recommendation

Apply `yzUSD` pause and redeem restriction enforcement consistently for all paths that redeem via `PSM`.

Consider one of the following approaches based on the intended enforcement behavior:

- Enforce checks inside `PSM._redeem()`
  - Before burning `yzUSD`, verify `yzUSD` is not paused and the redeemer is eligible under redeem restrictions.
  - This makes `_redeem()` self-contained and avoids future bypasses if new entry points are added.
- Validate at entry points (align checks with the user action)
  - In `PSM.redeem()`: validate pause/restrictions before calling `_redeem()`.
  - In `PSM.createRedeemOrder()`: consider validating redeemability at creation time to prevent unfillable orders when vaults are paused or restricted.
  - In `PSM.fillRedeemOrders()`: clarify whether pause and redeem restrictions should be enforced at fill time if the `yzUSD` state changes after order creation.

Low findings

[L-01] Dos in `withdrawLiquidity` due to frontrunning via instant withdrawals

Description

`withdrawLiquidity()` is used to withdraw tokens from the contract.

```
function withdrawLiquidity(uint256 assets, address receiver)
        external
        nonReentrant
        onlyRole(LIQUIDITY_MANAGER_ROLE)
    {
        _withdrawLiquidity(receiver, assets);
    }
```

A malicious actor can monitor the mempool for pending `withdrawLiquidity` transactions and front-run them, draining liquidity from the pool.

As a result, when the liquidity manager's `withdrawLiquidity` transaction is executed, it may fail due to an insufficient balance of the underlying asset. This effectively prevents the manager from withdrawing funds, causing denial of service (DOS).

Recommendation: To mitigate this issue, check whether the requested `assets` amount exceeds `balanceOf(address(this))`. If it does, transfer the full contract balance instead of reverting, ensuring the liquidity manager can still recover available funds.

[L-02] Dos vulnerability in `getPendingOrderIds` due to large `_pendingOrderIds` size

`_pendingOrderIds()` is used to retrieve all pending order IDs.

```
function getPendingOrderIds() external view returns (uint256[] memory) {
    uint256 length = _pendingOrderIds.length();
    uint256[] memory ids = new uint256[](length);
    for (uint256 idx = 0; idx < length; idx++) {
        ids[idx] = _pendingOrderIds.at(idx);
    }
    return ids;
}
```

If `_pendingOrderIds` grows too large, this function may exceed the block gas limit, causing the call to revert. As a result, it would become impossible to retrieve the pending orders until `_pendingOrderIds` is reduced.

This is problematic because this is the only function available to view pending orders. If pending orders need to be retrieved off-chain, it would be impossible to obtain them.

Recommendation: Introduce a paginated function that allows retrieving pending orders in batches.

[L-03] Emergency pause does not allow fund retrieval

Description

The current logical implementation suggests that mint and redeem must be paused during a paused state of the `YuzuILP.sol` contract.

However, during an emergency, users cannot complete or cancel their redeem orders. This paused state would not allow the admin to withdraw their funds from the platform, as the `rescueTokens()` explicitly does not allow withdrawing the asset token:

```
    /// @notice Rescue tokens from the contract
    function rescueTokens(address token, address to, uint256 amount) external
onlyRole(ADMIN_ROLE) {
        if (token == address(this)) {
            uint256 outstandingBalance = balanceOf(address(this)) - totalPendingOrderSize();
            if (amount > outstandingBalance) {
                revert ExceededOutstandingBalance(amount, outstandingBalance);
            }
        } else if (token == _asset) {                <<@
            revert InvalidAssetRescue(token);
        }
        SafeERC20.safeTransfer(IERC20(token), to, amount);
    }
```

It is recommended to separate the concerns of the emergency pause from a normal pause required for updating the pool via `updatePool()` by the pool manager.

[L-04] Parameter mismatch while throwing `UnauthorizedOrderFinalizer` error

Description

The `UnauthorizedOrderFinalizer()` error is defined to revert with `(account, receiver, controller)`:

```
interface IYuzuOrderBookDefinitions is IYuzuDefinitions {
    error InvalidZeroAddress();
    error FillWindowTooHigh(uint256 provided, uint256 max);
    error UnderMinRedeemOrder(uint256 tokens, uint256 min);
    error UnauthorizedOrderManager(address account, address owner, address controller);
    error UnauthorizedOrderFinalizer(address account, address receiver, address
controller);          <<@
    // . . .
```

However, the `YuzuOrderBook::finalizeRedeemOrder()` reverts with `order.owner`, which is inconsistent with the definition above:

```
    function finalizeRedeemOrder(uint256 orderId) public virtual {
        address caller = _msgSender();
        Order storage order = _getOrder(orderId);
        if (caller != order.owner && caller != order.controller) {
            revert UnauthorizedOrderFinalizer(caller, order.owner, order.controller);
<<@
        }
        // . . .
```

It is recommended to fix the error signature with address `owner`.

[L-05] Missing prechecks in `PSM.createRedeemOrder` allow unfillable redeem orders

Description

`PSM.createRedeemOrder()` does not validate whether a redeem order is fillable at creation time. It blindly accepts orders and transfers `syzUSD` shares to the `PSM` even when redemption is currently disabled.

This differs from the redeem order creation flow in `yzUSD` (`createRedeemOrder()`) and `syzUSD` (`initiateRedeem()`), which both guard order creation via `maxRedeemOrder()` checks.

PSM.sol#L234-L255

```
function _createRedeemOrder(address _owner, address receiver, uint256 shares) internal
returns (uint256) {
    if (receiver == address(0)) {
        revert InvalidZeroAddress();
    }

    uint256 orderId = _orderCount;
    _orders[orderId] = Order({
        shares: shares,
        owner: _owner,
        receiver: receiver,
        createdAt: SafeCast.toUint40(block.timestamp),
        status: OrderStatus.Pending
    });
    _orderCount++;
    // slither-disable-next-line unused-return
    _pendingOrderIds.add(orderId);

    SafeERC20.safeTransferFrom(IERC20(vault1()), _owner, address(this), shares);

    emit CreatedRedeemOrder(_owner, receiver, _owner, orderId, shares);
    return orderId;
}
```

As a result, users can create redeem orders in states where redemption is blocked, including:

1. `syzUSD` is paused or
2. `yzUSD` is paused or has `isRedeemRestricted == true` and the user is ineligible.

Because users cannot self-cancel orders, these orders become stuck and can only be resolved by an address with `ORDER_FILLER_ROLE`.

Consider adding validation in `PSM.createRedeemOrder()` to ensure redemption is currently allowed before accepting orders.

[L-06] Psm lacks `maxDeposit` and `maxRedeem` preventing users from querying limits

Description

The `PSM` contract lacks `maxDeposit()` and `maxRedeem()` functions, preventing users from querying the maximum amounts they can deposit or redeem before executing.

This is problematic because the `PSM` contract wraps two underlying vaults (expecting: `yzUSD` as `_vault0` and `syzUSD` as `_vault1`), each with their own restrictions that can cause operations to revert.

For deposits via `PSM.deposit()` (PSM.sol#L213-L221), the operation can revert when:

- `yzUSD` (`_vault0`) is paused, as `YuzuProto.maxDeposit()` returns 0
- `yzUSD` has `isMintRestricted` enabled and `PSM` lacks `MINTER_ROLE`
- `syzUSD` (`_vault1`) is paused, as `syzUSD.maxDeposit()` returns 0

For redemptions via `PSM.redeem()` (PSM.sol#L224-L232), the operation can revert when:

- `syzUSD` (`_vault1`) is paused, as `syzUSD.maxDeposit()` returns 0.

Consider implementing `maxDeposit()` and `maxRedeem()` in the `PSM` contract that aggregates the constraints from both underlying vaults.

[L-07] Forced cancellations are limited by the `whenNotPaused` modifier

Description

`YuzuProtoV2.sol` overrides the `cancelRedeemOrder()` function to explicitly skip the `caller` and `dueTime` validation if `hasRole(ORDER_FILLER_ROLE, caller)` returns `true`.

However, the `_cancelRedeemOrder()` function used is the overridden implementation from `YuzuProto.sol`, which uses the `whenNotPaused` modifier:

```
    function _cancelRedeemOrder(Order storage order) internal virtual override whenNotPaused
{
        super._cancelRedeemOrder(order);
    }
```

This behavior is inconsistent with other admin-exposed (`ORDER_FILLER_ROLE`) functionalities and restricts the availability of force cancellations baselessly.

Consider bypassing the `whenNotPaused` modifier for the force cancel path. For instance:

```
-       _cancelRedeemOrder(order);
+       if (paused() && !hasRole(ORDER_FILLER_ROLE, caller)) {
+           revert Paused();
+       }
+       YuzuOrderBook._cancelRedeemOrder(order);
```

[L-08] Potential DoS in `YuzuILPV2` due to unconsolidated redeem order fill

Description

After the upgrade, the `YuzuILP._fillRedeemOrder()` starts decreasing the `poolSize` by the `_discountYield()` deposit amount calculated based on `assets + fee` that includes the redeemer's pro-rata share of the currently ongoing distribution (the new distribution started with `YuzuILPV2.distribute()`). It is necessary because `YuzuILPV2._totalAssets()` accounts for the total value of that distribution by adding `_distributedAssets()` to the return value - which returns the assets distributed based on distribution parameters, which does not change on redemptions - failing to do so would effectively double-count that redeemer's pro-rata share in distributed assets in the `totalAssets()` until `distribute()` is called again.

Therefore, by design, as long as distribution is unconsolidated, the `_discountYield(assets + fee)` that we subtract from `poolSize` is larger by the redeemer's distributed assets share than the actual redeemer's pro-rata share in `poolSize` at that time. Usually, this has no impact as this state is cleared after the next `distribute()` function call.

However, in an event where all redeemers wish to exit the protection pool, the last one or few redeemers' redemption orders may be unfillable until `distribute()` is called; otherwise, reverting on underflow. This causes a temporary DoS of redemptions for last redeemers and forces the pool manager to terminate ongoing distribution and start a new one with arguments set to `distribute(0,1)` - to ensure the DoS is eliminated. Whether last redemptions are affected depends on their total %-share in the total supply and the size of the distribution. The likelihood grows if this scenario is preceded by an `updatePool()` that decreases `poolSize` (protocol loss consolidated) because it reduces `poolSize` without reducing the distributed-assets component of what we subtract.

To avoid complex code changes, acknowledge the possibility of such scenarios occurring during mass withdrawal and ensure the `POOL_MANAGER_ROLE` swiftly calls `terminateDistribution()` (if in progress) and then `distribution(0, 1)` to make sure all redemptions are fillable.

[L-09] EIP-712 domain version not updated to V2 allowing old signatures to remain valid

Description

`StakedYuzuUSDV2` and `YuzuILPV2` are upgraded implementations of the original `StakedYuzuUSD` and `YuzuILP` contracts.

The new version correctly uses a reinitializer to mark the upgrade.

```
function reinitialize() external reinitializer(2) {}
```

However, the original contracts initialize an EIP-712 domain during V1 initialization.

```
__EIP712_init(__name, "1");
```

In `StakedYuzuUSDV2` and `YuzuILPV2`, the EIP-712 domain is not re-initialized with a new version. As a result, the EIP-712 domain separator remains unchanged after the upgrade.

Because the proxy address, contract name, `block.chainid`, and EIP-712 version (`"1"`) remain the same, signatures generated for V1 remain valid in V2.

This creates a cross-version replay risk. A signature created by an owner for use in V1 can be used in V2. Since V2 introduces new logic and execution paths, such a signature may authorize actions that were not possible or intended in V1, leading to unexpected or unsafe behavior.

Recommendations

Re-initialize the EIP-712 domain during the V2 upgrade. This ensures that all V1 signatures are invalidated and cannot be used in V2.

```
function reinitialize() external reinitializer(2) {
+     __EIP712_init(name(), "2");
}
```